

Memoria Técnica

cv-lit

Sistema de Monitorización Automática
de Línea de Costa

Guardamar del Segura · Alicante · España

Tech4D Lab
Universidad de Alicante
nickhernd

Junio 2026

Resumen

cv-lit es un sistema software de monitorización automática de línea de costa mediante visión por computador, desarrollado para el Ayuntamiento de Guardamar del Segura en colaboración con el Instituto de Ecología Litoral (IEL) y la Universidad de Alicante.

El sistema procesa imágenes oblicuas de **6 cámaras fijas Obscape** instaladas a lo largo del frente litoral (tramo norte-sur $\approx 3,5$ km), extrayendo automáticamente la línea de costa y proyectándola a coordenadas geográficas UTM EPSG:25830 (ETRS89 / UTM zona 30N).

Esta memoria recoge la arquitectura completa del sistema, los algoritmos de procesamiento de imagen (SIFT, RANSAC, SAM), el módulo de alineación masiva no-destructiva diseñado en la iteración de junio de 2026, la interfaz web de validación por lotes, los resultados de calibración geométrica obtenidos para las seis cámaras, y la documentación operacional completa para el despliegue y mantenimiento del sistema.

Índice general

1. Descripción del Proyecto	6
1.1. Contexto y motivación	6
1.2. Partes implicadas	6
1.3. Objetivos del sistema	6
1.4. Restricciones no negociables	7
1.5. Entregables	7
2. Infraestructura de Cámaras	8
2.1. Sistema Obscape	8
2.2. Distribución geográfica	8
2.3. Descarga automática de imágenes	8
2.3.1. Automatización con Cron	9
3. Arquitectura del Sistema	10
3.1. Stack tecnológico	10
3.2. Estructura de directorios	11
3.3. Pipeline de procesamiento	12
4. Calibración Geométrica	13
4.1. ¿Qué es la calibración y para qué sirve?	13
4.2. De dónde salen los puntos de control (GCPs)	14
4.3. Flujo actual: calibración interactiva desde la interfaz web	14
4.4. Estimación con RANSAC	15
4.4.1. Aviso de geometría inestable	16
4.5. Dos umbrales que no hay que confundir	16
4.6. Métricas del ajuste: RMSE y MAE	17
4.7. Resultados de calibración (campana junio 2026)	17
4.8. Imágenes de diagnóstico	18
4.9. Comparación con la homografía de alineación (capítulo 5)	19
5. Módulo de Alineación Masiva	20
5.1. Motivación	20
5.2. Arquitectura de estados	20
5.2.1. Estructura del directorio temporal por job	21
5.3. Pipeline de alineación: algoritmo	21
5.3.1. Parámetros configurables	21
5.3.2. Estrategia de dos pasadas	22
5.3.3. Flujo completo del pipeline	22
5.4. Renderizado del mapa de delta	23

5.4.1.	Capa 1 — Heatmap de diferencia absoluta	23
5.4.2.	Capa 2 — Divergencia de bordes (overlay cian)	23
5.4.3.	Capa 3 — Métrica escalar δ_{px} (flujo óptico Farneback)	23
5.5.	API REST del módulo	24
5.6.	Modelos de dominio	24
5.7.	Flujo de commit (two-phase write)	25
6.	Máscaras de Zonas Estables SIFT	26
6.1.	Problema: objetos dinámicos en SIFT	26
6.2.	Solución: máscaras de regiones estables	26
6.3.	Zonas por cámara	27
6.4.	Visualización de zonas estables por cámara	27
6.5.	Corrección CAM_1: problema de gaviotas	30
6.5.1.	Coordenadas en píxeles (imagen 4608×2682)	30
6.6.	Cómo ajustar una máscara	30
7.	Interfaz Web	32
7.1.	Arquitectura frontend	32
7.2.	Componente BatchAlignment.vue	32
7.2.1.	Variables de estado	32
7.2.2.	Props y eventos	33
7.2.3.	Layout de la fase reviewing	33
7.2.4.	Indicadores del filmstrip	33
7.2.5.	Polling de progreso	34
7.3.	Flujo completo de uso	34
8.	Región de Interés (ROI)	35
8.1.	Criterios de delimitación	35
8.2.	Coordenadas por cámara	35
9.	Segmentación y Extracción de Línea de Costa	37
9.1.	Segmentación semántica con SAM	37
9.2.	Extracción de línea de costa	38
9.3.	Proyección a EPSG:25830	38
9.4.	Exportación GeoJSON	38
9.5.	Índice de confianza (Confianza_IA) y una limitación conocida	39
10.	Validación y Métricas de Calidad	41
10.1.	Estrategia de validación	41
10.2.	Métricas	41
10.3.	Criterios de aceptación	42
10.4.	Factores de error a monitorizar	42
10.5.	Validación en QGIS	42
11.	Módulo de Automatización (Auto Mode)	43
11.1.	¿Qué problema resuelve?	43
11.2.	Requisito previo: la cámara debe estar ya calibrada	43
11.3.	Las cuatro fases	44
11.4.	Por qué el commit automático es seguro sin revisión humana	45

11.5. Modelos de dominio	45
11.6. API REST del módulo	46
11.7. Nota de empaquetado: import dinámico y PyInstaller	46
12.Estado del Proyecto y Roadmap	47
12.1. Estado actual (agosto 2026)	47
12.2. Completado en la iteración de junio 2026	47
12.3. Fase 3 — Automatización y robustez (completada)	48
12.4. Fase 4 — Entrega e integración (completada)	48
12.5. Ronda de pulido para producción (julio-agosto 2026)	48
13.Guía de Operación	50
13.1. Inicio rápido	50
13.2. Descarga de imágenes	50
13.3. Calibración	50
13.4. Alineación masiva (CLI)	51
13.5. Pipeline de extracción	51
13.6. Operación del día a día: Auto Mode	51
13.7. Aplicación de escritorio (sin entorno de desarrollo)	52
14.Instalación y Configuración	53
14.1. Requisitos del sistema	53
14.2. Instalación del entorno Python	53
14.3. Instalación del frontend	54
14.4. Configuración de la API Obscape	54
14.5. Generar el instalador de Windows	54
A. Convenciones de código y Git	56
A.1. Estrategia de ramas	56
A.2. Nomenclatura de ficheros	56
A.3. Proyección obligatoria	56
B. Glosario	57
C. Historial de cambios	59

Índice de figuras

3.1. Pipeline completo de procesamiento cv-lit	12
4.1. Los tres pasos del asistente de calibración implicados en este capítulo .	14
4.2. Diagnóstico de calibración — reproyección de GCPs (vectores $\times 10$) . .	18
5.1. Máquina de estados del módulo BatchAlignment	21
5.2. Flujo de commit no-destructivo	25
6.1. CAM 1 — horizonte del mar ($y = 0,39\text{--}0,48$) + arena baja izq + vegetación dcha (hasta fondo). Zona diseñada para evitar las gaviotas que vuelan en $y < 0,38$	28
6.2. CAM 2 — horizonte del mar ($y = 0,34\text{--}0,48$) + zona izquierda con vallas y vegetación ($x = 0,00\text{--}0,28$, hasta el borde inferior). Las vallas y masa vegetal ofrecen features estables cuando la visibilidad no permite distinguir el Peñón de Ifach.	28
6.3. CAM 3 — horizonte del mar ($y = 0,45\text{--}0,56$) + kiosco de playa en la esquina inferior derecha ($x = 0,87\text{--}1,00$, $y = 0,80\text{--}0,93$). Se excluye deliberadamente la franja $x = 0,72\text{--}0,87$: las palmeras que hay ahí agitan sus hojas con el viento y generaban correspondencias SIFT inestables (rotaciones espurias de más de 100° en las pruebas de alineación). . . .	28
6.4. CAM 4 — horizonte del mar ($y = 0,38\text{--}0,52$) + paseo marítimo/edificios en la esquina izquierda ($x = 0,00\text{--}0,28$, hasta el borde inferior). La infraestructura urbana proporciona features angulares muy discriminantes.	29
6.5. CAM 5 — horizonte del mar ($y = 0,36\text{--}0,50$) + franja derecha estrecha ($x = 0,68\text{--}1,00$). La zona derecha captura vegetación y estructuras fijas con mayor estabilidad que la franja izquierda anterior.	29
6.6. CAM 6 — horizonte del mar ($y = 0,40\text{--}0,54$) + bloque de edificios en la esquina izquierda ($x = 0,00\text{--}0,35$). Los edificios tienen fachadas con ventanas y balcones que generan features muy estables.	29
6.7. Comparativa de máscaras SIFT para CAM 1 (imagen 4608×2682 px) .	30
11.1. Pipeline de Auto Mode — cada fase reutiliza código ya existente, sin reimplementar nada	44

Índice de cuadros

1.1. Actores del proyecto	6
1.2. Atributos del GeoJSON de salida	7
2.1. Estaciones Obscape en Guardamar del Segura	8
3.1. Dependencias principales	10
4.1. Umbral por varilla (píxeles) frente a umbral de aceptación (metros) . .	16
4.2. Resultados de precisión por cámara — campaña de calibración inicial .	17
4.3. Las dos homografías del sistema, lado a lado	19
5.1. Parámetros del pipeline de alineación (<code>batch_alignment.py</code>)	21
5.2. Leyenda del heatmap HOT	23
5.3. Endpoints del router <code>/api/batch/</code>	24
6.1. Configuración de máscaras SIFT por cámara	27
6.2. Conversión fracción → píxeles para CAM_1	30
7.1. Props del componente BatchAlignment	33
7.2. Código de color en el filmstrip	33
8.1. ROIs de producción por cámara (<code>calibration/roi_config.json</code>)	35
10.1. Umbrales de calidad del sistema	42
11.1. Endpoints del router <code>/api/automode/</code>	46
12.1. Estado de hitos del proyecto	47
14.1. Requisitos hardware y software	53
14.2. Parámetros de conexión Obscape	54
C.1. Historial de versiones del sistema	61

Capítulo 1

Descripción del Proyecto

1.1 Contexto y motivación

La playa de Guardamar del Segura es un espacio de alto valor ecológico y turístico sujeto a procesos de erosión y acreción que requieren seguimiento sistemático. La medición manual de la línea de costa mediante levantamientos GNSS es costosa, puntual y no escala para cubrir todo el frente litoral con frecuencia horaria.

El proyecto **cv-lit** surge para automatizar esta medición aprovechando la infraestructura de cámaras fijas ya desplegada por el servicio de vigilancia de la playa, convirtiendo cada fotografía en un dato georreferenciado de la posición de la interfaz arena-mar.

1.2 Partes implicadas

Cuadro 1.1: Actores del proyecto

Rol	Entidad
Cliente	Ayuntamiento de Guardamar del Segura
Datos GNSS (GCPs)	Instituto de Ecología Litoral (IEL)
Desarrollo	1 ingeniero a tiempo parcial (Tech4D Lab, UA)
Supervisión	Universidad de Alicante
Duración	6 meses (enero – junio 2026)

1.3 Objetivos del sistema

El producto final es un **GeoJSON proyectado en EPSG:25830** con los siguientes atributos obligatorios por cada detección:

Cuadro 1.2: Atributos del GeoJSON de salida

Campo	Tipo	Descripción
ID_Camara	String (CAM_1".."CAM_6")	Identificador de cámara origen
Timestamp	ISO 8601 UTC	Fecha y hora de captura
Confianza_IA	Float (0–1)	Score de confianza del modelo SAM
Area_Seca_m2	Float	Superficie de playa seca (m ²)
RMSE_m	Float	RMSE de la calibración activa en el momento del análisis
n_puntos	Integer	Número de puntos de la polilínea de costa

1.4 Restricciones no negociables

- Toda implementación **debe** producir GeoJSON en **EPSG:25830**.
- Fase horaria prioritaria: **12:00h solar local**.
- Modelo de segmentación: **SAM** (Segment Anything Model, Meta AI).
- Cada cámara tiene su propio perfil de calibración *independiente*.
- Criterio de aceptación de precisión: **RMSE < 2 m** en condiciones estándar (ver RMSE_GOOD_M, capítulo 4).

1.5 Entregables

1. Instalador de escritorio para Windows (**LineaDeCosta-Setup.exe**), configurado y documentado (esta memoria) — ver capítulo 14.
2. Perfiles de calibración por cámara (6 ficheros **.npz** / **.json**).
3. Interfaz web de operación y validación por lotes (la misma interfaz sirve tanto en modo navegador como empaquetada dentro de la aplicación de escritorio).
4. Informe de validación con métricas RMSE y MAE, exportable en PDF por cámara.

Capítulo 2

Infraestructura de Cámaras

2.1 Sistema Obscape

Las seis cámaras están gestionadas por la plataforma Obscape y se acceden via API REST autenticada. La tabla 2.1 lista los parámetros de registro de cada estación.

Cuadro 2.1: Estaciones Obscape en Guardamar del Segura

Cámara	Station	Referencia	Latitud (°N)	Longitud (°E)	Última act.
CAM 1	8213	PTM61471	38.110327	−0.643027	20-05-2026
CAM 2	8214	PTM61474	38.096291	−0.645725	20-05-2026
CAM 3	8212	PTM61473	38.087348	−0.646994	20-05-2026
CAM 4	8211	PTM61475	38.087385	−0.647078	20-05-2026
CAM 5	8209	PTM61472	38.076774	−0.648117	20-05-2026
CAM 6	8210	PTM61470	38.076796	−0.648114	20-05-2026

Nota geográfica

CAM 3 y CAM 4 son casi coincidentes ($\Delta \approx 8$ m) y cubren el mismo tramo desde ángulos ligeramente distintos. Lo mismo ocurre con CAM 5 y CAM 6. Esto permite comparación cruzada de la línea detectada para validación interna.

2.2 Distribución geográfica

Las cámaras se distribuyen de norte a sur a lo largo de $\approx 3,5$ km de frente litoral. La cobertura solapada garantiza continuidad en la extracción de línea de costa sin lagunas.

2.3 Descarga automática de imágenes

El módulo `acces_api/scheduled_download.py` implementa una **ventana deslizante de 14 días** que garantiza recuperación automática ante fallos puntuales de descarga:

Descarga estándar

```
1 # Últimas 2 semanas, hora solar 12:00 (por defecto)
2 python3 acces_api/scheduled_download.py
3
4 # Personalizar ventana y hora
5 python3 acces_api/scheduled_download.py --days 30 --hour 15
6
7 # Descarga masiva histórica (todas las horas)
8 python3 acces_api/scheduled_download.py --days 90 --hour all
```

El sistema evita duplicados comprobando la existencia del fichero antes de descargar y utiliza un nombre determinista: `UNIX_YYYYMMDD_HHMMSS_REF.jpg`.

2.3.1 Automatización con Cron

Para descarga diaria autónoma:

`/etc/cron.d/cvlit-download`

```
1 # Descarga diaria a las 13:00h (12h solar + margen de publicación API)
2 0 13 * * * cd /home/nickhernd/Desktop/cv-lit && \
3 python3 acces_api/scheduled_download.py >> data/logs/cron.log 2>&1
```

Capítulo 3

Arquitectura del Sistema

3.1 Stack tecnológico

Cuadro 3.1: Dependencias principales

Componente	Versión	Rol
Python	3.11	Runtime principal
FastAPI	≥ 0.110	API REST backend
OpenCV	4.x	Procesamiento de imagen
NumPy/SciPy	–	Álgebra matricial
SAM (Meta AI)	–	Segmentación semántica
PyTorch	≥ 2.0	Backend SAM
Vue 3 + Vite	–	Interfaz web
Tailwind CSS	3.x	Estilos
fpdf2	–	Generación de informes PDF de calibración
PyInstaller + Inno Setup	–	Empaquetado del instalador de escritorio

¿Por qué no hay GDAL ni pyproj en la lista?

El sistema no reproyecta entre sistemas de referencia distintos en tiempo de ejecución: las coordenadas UTM de las varillas de calibración ya vienen dadas en EPSG:25830 (capítulo 4), así que la homografía píxel→UTM que calcula `calculate_homography()` proyecta directamente a ese sistema — `"EPSG:25830"` se escribe como metadato fijo en el GeoJSON de salida (`georef_export.py`), no se calcula con una librería de transformación de CRS. Si en el futuro hiciera falta soportar más de un huso UTM o reproyectar a otro sistema, ahí sí haría falta pyproj.

3.2 Estructura de directorios

Árbol del repositorio

```

1  cv-lit/
2  +-- acces_api/           # Módulo Obscape API
3  |   +-- obscape_api.py   # Cliente REST
4  |   +-- scheduled_download.py
5  +-- backend/            # Servidor FastAPI
6  |   +-- main.py          # App principal + endpoints de cámaras/calibración
7  |   +-- batch_alignment.py # Pipeline de alineación masiva
8  |   +-- auto_mode.py     # Auto Mode: descarga + alineación + análisis
9  |   +-- config.py        # Constantes globales + CAMERAS
10 |   +-- desktop_launcher.py # Punto de entrada de la app de escritorio
    ↳ (pywebview)
11 |   +-- desktop_launcher.spec # Spec de PyInstaller para el .exe principal
12 |   +-- download_sam.py    # Descargador aparte del checkpoint SAM (opcional)
13 |   +-- download_sam.spec  # Spec de PyInstaller para download_sam.exe
14 +-- installer/           # Instalador de escritorio para Windows
15 |   +-- cv-lit.iss         # Script de Inno Setup
16 |   +-- output/           # LineaDeCosta-Setup.exe (generado, excl. Git)
17 +-- calibration/         # Perfiles y máscaras por cámara
18 |   +-- cam_N_H.npy        # Matriz H 3×3 en float64
19 |   +-- cam_N_profile.json # Metadatos, RMSE, nombre de perfil, operador, notas
20 |   +-- roi_config.json    # ROIs por cámara
21 |   +-- alignment_masks.json # Zonas SIFT estables
22 +-- data/                # Imágenes y resultados (excl. Git) - CVLIT_DATA_DIR
23 |   +-- camera1/ ... camera6/ # una carpeta por cámara (ver
    ↳ CAMERAS[i]["folder"])
24 |   |   +-- *.jpg          # capturas originales, nombradas
    ↳ <epoch>_<AAAAMDD>_<HHMMSS>_<serie>.jpg
25 |   |   +-- aligned/      # salida del commit de alineación masiva
    ↳ (batch_alignment.py)
26 |   +-- CAM_1/ ... CAM_6/
27 |   |   +-- json/         # anotaciones de varillas (una por imagen marcada)
28 |   +-- system_logs.jsonl  # buffer de actividad que sirve GET /api/logs
29 +-- docs/                # Documentación técnica
30 |   +-- memoria.tex       # Esta memoria [NUEVO]
31 |   +-- reports/
32 +-- frontend/            # Interfaz Vue 3
33 |   +-- src/
34 |       +-- App.vue        # Layout raíz + navegación
35 |       +-- api.js         # API_BASE (VITE_API_URL) usado por todos los
    ↳ componentes
36 |       +-- components/
37 |           +-- Dashboard.vue # Vista general del sistema
38 |           +-- ImageIngest.vue # Carga de imágenes
39 |           +-- Calibration.vue # Wizard de calibración (7 pasos)
40 |           +-- BatchAlignment.vue # Alineación masiva
41 |           +-- AutoMode.vue # Auto Mode
42 |           +-- CoastlineAnalysis.vue # Resultados de análisis

```

```

43 |         +-- GeoJSONMap.vue      # Mapa GeoJSON exportado
44 |         +-- Map.vue            # Mapa base reutilizable
45 |         +-- Cameras.vue        # Gestión de cámaras
46 +-- homography/
47 |     +-- align_images.py        # CLI standalone de alineación
48 +-- proces_images/              # Algoritmos de procesamiento
49 |     +-- extract_coastline.py
50 |     +-- segmentation_sam.py
51 |     +-- georef_export.py
52 +-- scripts/                    # Herramientas de operación
53 +-- tests/

```

3.3 Pipeline de procesamiento

La figura 3.1 muestra el grafo completo de procesamiento desde la adquisición hasta la exportación del GeoJSON georreferenciado.

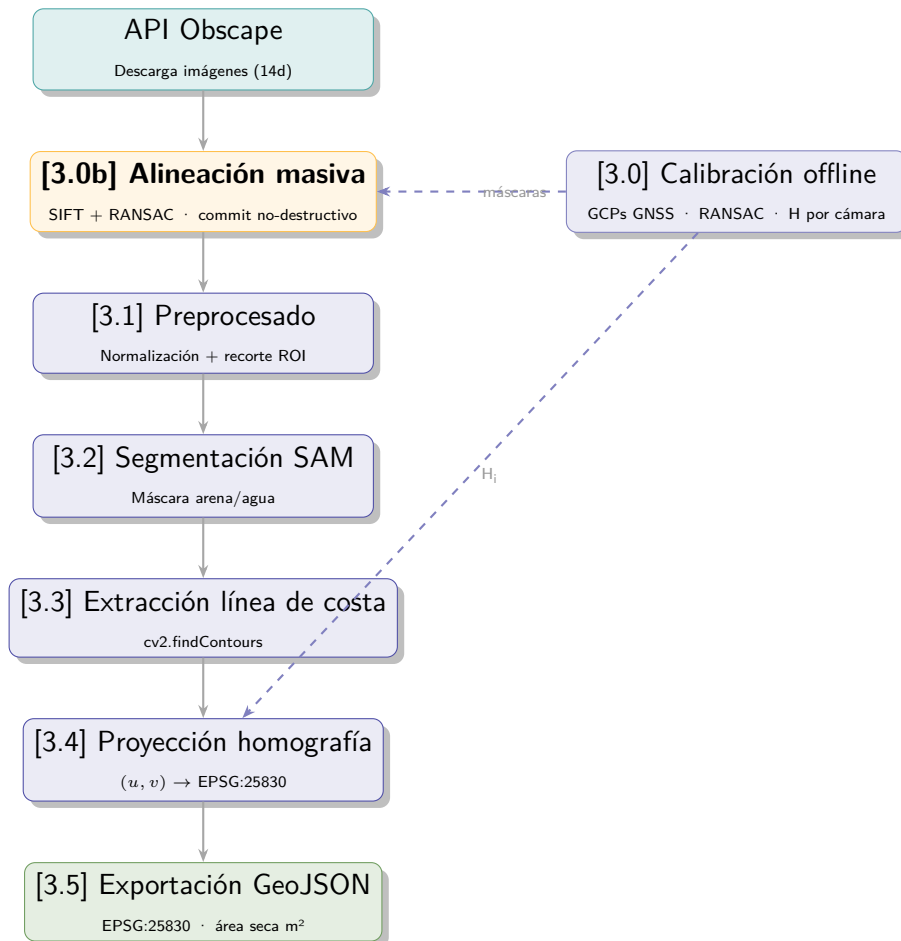


Figura 3.1: Pipeline completo de procesamiento cv-lit

Capítulo 4

Calibración Geométrica

4.1 ¿Qué es la calibración y para qué sirve?

Cada cámara ve la playa en **perspectiva**: un píxel de la imagen no dice, por sí solo, qué punto real de la playa representa. La calibración resuelve exactamente ese problema: encuentra la transformación matemática que convierte un píxel (u, v) de la foto en una coordenada real (X, Y) en metros, en el sistema de referencia EPSG:25830 (ETRS89 / UTM huso 30N).

Esa transformación es una **homografía** $\mathbf{H}$, una matriz 3×3 :

$$\begin{pmatrix} X \\ Y \\ 1 \end{pmatrix} \sim \mathbf{H} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \quad (4.1)$$

¿Por qué una homografía y no algo más simple?

Una homografía es la transformación correcta cuando se proyecta un plano (la arena de la playa, que se asume aproximadamente plana) visto desde un punto de vista arbitrario — mezcla en una sola matriz la rotación, traslación, escala y la perspectiva de la cámara. Un simple factor de escala metros/píxel no sirve aquí: un píxel cerca de la cámara cubre muchísimo menos terreno real que un píxel cerca del horizonte, y esa relación cambia de forma no lineal con la distancia — es exactamente lo que la homografía sí modela.

Ojo: esta NO es la única homografía del sistema

Este capítulo cubre la homografía de **calibración geométrica**: píxel de la foto $\rightarrow$ coordenada UTM real. Existe una **segunda homografía**, completamente distinta, en el módulo de alineación masiva (capítulo 5): esa es *imagen* $\rightarrow$ *imagen* (corrige el pequeño desplazamiento de la cámara entre dos fotos) y no tiene nada que ver con coordenadas UTM. Las dos se calculan con la misma función de OpenCV (`cv2.findHomography` con RANSAC) pero resuelven problemas distintos, sobre puntos de entrada distintos, y no se deben confundir. Ver la nota comparativa en §4.9.

4.2 De dónde salen los puntos de control (GCPs)

La homografía se ajusta a partir de **GCPs** (*Ground Control Points*, aquí llamados “varillas”): puntos de los que se conocen *las dos* coordenadas del mismo punto físico — dónde cae en la foto (píxel) y dónde está en la realidad (UTM, en metros). Hacen falta un mínimo de 4 pares para que el sistema tenga suficiente información (una homografía tiene 8 grados de libertad); en la práctica se recomiendan 8 o más, y cuantos más se usen, más robusto es el ajuste frente a un punto puntual mal marcado.

Cada GCP llega al sistema por una de estas dos vías:

- **Manual:** clic directo sobre la imagen en la interfaz web (paso *Marcación*, §4.3), con una lupa de precisión ($\times 6$) para acertar el píxel exacto en fotos de varios miles de píxeles de ancho.
- **Importado de un levantamiento de campo** (CSV con columna FILE que liga cada varilla a la sesión de disparo en la que se topografió): el píxel llega como *aproximado* (varilla “pendiente”, marcador ámbar) y hay que confirmarlo con la lupa antes de que cuente para el ajuste — así un píxel mal ubicado en el CSV no puede colar error silenciosamente en la calibración.

¿El cálculo se hace sobre una imagen o sobre todas las de la cámara?

Sobre **una sola imagen**, la que esté seleccionada en ese momento en el paso *Marcación* —cada imagen tiene su propio conjunto de varillas marcadas y su propia homografía calculada, guardada junto a esa imagen concreta (`data/CAM_X/json/{imagen}.json`, campo *calibration*)—. Lo que SÍ es por cámara es el *resultado activo*: cada vez que se calcula (o recalcula) la homografía de una imagen, esa **H** se copia también al **perfil de la cámara** (`calibration/cam_X_profile.json` y `cam_X_H.npy`), sobrescribiendo la anterior. Es esa copia a nivel de cámara —la última calibración calculada, sea de la imagen que sea— la que usa el análisis de línea de costa (capítulo 10) para *todas* las imágenes nuevas de esa cámara, hasta la siguiente recalibración. En otras palabras: se calibra imagen a imagen, pero solo hace falta una calibración vigente por cámara para poder analizar el resto de sus capturas.

4.3 Flujo actual: calibración interactiva desde la interfaz web

La calibración se hace hoy **enteramente desde el navegador** (pantalla *Calibración*), no con scripts de línea de comandos. El asistente tiene 7 pasos; los tres que corresponden a este capítulo son *Marcación*, *Cálculo* y *Validación*:

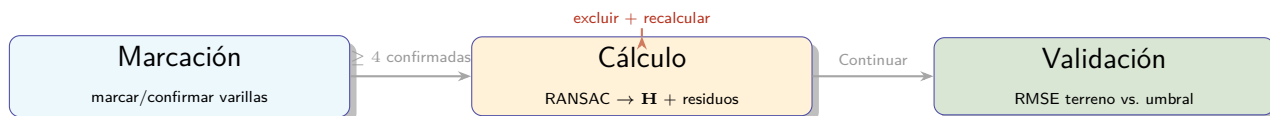


Figura 4.1: Los tres pasos del asistente de calibración implicados en este capítulo

1. **Marcación:** se marca (o confirma) cada varilla sobre la foto. Un clic directo encima

del marcador de una varilla pendiente la confirma en su sitio; un clic en cualquier otro punto de la imagen la reposiciona. Cada varilla confirmada se pinta en **verde**; las pendientes, en **ámbar**.

2. **Cálculo:** al pulsar *Calcular homografía* se ejecuta el ajuste (§4.4) y aparece una tabla de error de reproyección por varilla — se puede excluir una varilla concreta y recalcular sin ella, de forma iterativa, antes de pasar a Validación. Como la calibración es por imagen (§4.3), este paso incluye también un selector para pinchar entre las distintas fotos de la cámara y comparar el resultado de cada una sin volver a Marcación; la última que se calcule es la que queda activa para la cámara. Cada fila de la tabla muestra, además del error, el **píxel y la coordenada UTM tal y como se marcaron** — para poder verificar cada varilla contra el CSV/Excel de origen si un valor no encaja como se esperaba — y, al pasar el ratón sobre el error, el valor *predicho* por la homografía en ese mismo punto, para ver exactamente qué discrepancia produce el error.
3. **Validación:** resumen final de la calidad de la calibración de esta imagen — RMSE en metros frente al umbral objetivo, y estado *Óptimo/Revisar*.

Scripts de calibración retirados

Versiones anteriores de este sistema incluían herramientas de línea de comandos (`calibration_tool.py`, `automate_homography.py`, `recalibrate.py`) para marcar GCPs fuera del navegador. Se han retirado del repositorio: la calibración de producción vive por completo en el backend web (`calculate_homography()`, invocado desde el paso Cálculo) y esas herramientas habían quedado desincronizadas de ese flujo real — `calibration_tool.py` en concreto ni siquiera llegaba a compilar. El flujo de este capítulo es el único vigente.

4.4 Estimación con RANSAC

Estimación de homografía con RANSAC -- `calculate_homography()` en el backend

```

1 H, ransac_mask = cv2.findHomography(
2     pts_px,                # píxeles marcados/confirmados en la foto
3     pts_utm,               # coordenadas UTM del catálogo de varillas
4     cv2.RANSAC,
5     3.0                   # umbral de reproyección en el dominio UTM: 3 metros
6 )

```

¿Qué hace RANSAC aquí y qué es un *inlier*?

RANSAC (*Random Sample Consensus*) prueba subconjuntos de varillas y se queda con la homografía que consigue que el mayor número posible de ellas encaje dentro de un margen de error tolerable (aquí, 3 m en UTM). Las varillas que SÍ encajan con esa homografía se llaman **inliers**; las que no —porque su UTM o su píxel están mal, o porque son incompatibles con el resto— quedan como *outliers* y no participan en el ajuste final. Es la misma idea que un “voto mayoritario” geométrico: una sola

varilla mal introducida no puede arruinar la homografía si el resto de varillas son consistentes entre sí. La tabla de residuos del paso Cálculo marca con un ✓ verde las varillas que RANSAC aceptó como inlier y con una ✗ roja las que descartó.

4.4.1 Aviso de geometría inestable

Si las varillas marcadas quedan casi todas alineadas a lo largo de una única línea (por ejemplo, una fila de estacas topografiadas siguiendo la orilla, con muy poca variación en la dirección perpendicular), el sistema queda **numéricamente mal condicionado**: existen infinitas homografías que explican igual de bien esos puntos, y una homografía así puede fallar estrepitosamente (RMSE de decenas de metros) aunque el cálculo en sí no produzca ningún error. Se detecta comprobando qué fracción de las varillas activas quedó como inlier de RANSAC:

$$f_{\text{inlier}} = \frac{\text{varillas inlier}}{\text{varillas activas}} \quad f_{\text{inlier}} < 0,6 \Rightarrow \text{aviso de geometría inestable} \quad (4.2)$$

Cuando esto ocurre, la interfaz muestra explícitamente cuántas varillas de cuántas encajaron y recomienda añadir varillas repartidas en más de una zona/profundidad de la imagen, en vez de dejar solo un RMSE alto sin explicación.

4.5 Dos umbrales que no hay que confundir

La pantalla de calibración maneja **dos umbrales distintos**, en dominios distintos, con propósitos distintos — es la fuente de confusión más habitual al leer los resultados:

Cuadro 4.1: Umbral por varilla (píxeles) frente a umbral de aceptación (metros)

	Umbral	Para qué sirve
Error por varilla	5 px (ajustable)	Ayuda visual para <i>localizar</i> qué varilla concreta podría estar mal puesta (un misclick, o una varilla del catálogo equivocada). No decide si la calibración es válida.
RMSE terreno	2 m (RMSE_GOOD_M)	Criterio real de aceptación de la calibración de esta imagen — el mismo valor que pondera el índice de confianza del análisis de línea de costa (capítulo 10).

Por qué el estado ya NO depende de que todas las varillas bajen del umbral en píxeles

En una iteración anterior de este sistema, el indicador “Óptimo / Revisar” exigía que *todas* las varillas individuales tuvieran un error de reproyección por debajo de 1 px. Con fotos de varios miles de píxeles de ancho marcadas a mano, ese criterio era prácticamente imposible de cumplir incluso con una calibración objetivamente

buena: se observó un caso real con RMSE terreno = 0,24 m (muy por debajo del objetivo de 2 m) donde 7 de 8 varillas aparecían igualmente como “sobre umbral”. El estado ahora se decide por el RMSE en metros frente a `RMSE_GOOD_M`, que es la métrica que de verdad importa para georreferenciar la línea de costa; el umbral en píxeles sigue existiendo solo como ayuda de diagnóstico por varilla.

4.6 Métricas del ajuste: RMSE y MAE

Para cada varilla activa (no excluida, no pendiente) se calcula el error de reproyección en los dos dominios:

$$e_{\text{px},i} = \left\| \mathbf{H}^{-1}(\text{UTM}_i) - \text{píxel}_i \right\| \quad (4.3)$$

$$e_{\text{m},i} = \left\| \mathbf{H}(\text{píxel}_i) - \text{UTM}_i \right\| \quad (4.4)$$

y a partir de esos residuos, el RMSE (penaliza más los errores grandes — útil para detectar varillas mal marcadas) y el MAE (distancia media, sin elevar al cuadrado):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2} \quad \text{MAE} = \frac{1}{N} \sum_{i=1}^N e_i \quad (4.5)$$

4.7 Resultados de calibración (campana junio 2026)

Cuadro 4.2: Resultados de precisión por cámara — campana de calibración inicial

Cámara	GCPs	RMSE Cal. (m)	RMSE Val. (m)	RMSE Val. (px)	Estado
CAM 1	51	1.53	1.45	19.36	OK
CAM 2	39	2.64	3.45	21.09	Revisar
CAM 3	49	1.61	2.32	38.38	OK
CAM 4	49	2.78	3.06	100.01	Revisar
CAM 5	61	1.45	1.38	14.81	OK
CAM 6	69	6.14	4.74	130.62	Critico

Cámaras con error elevado

Los errores de CAM 4 y CAM 6 superan ampliamente el objetivo de 2 m y son consistentes con el problema descrito en §4.4.1: en la sesión de campo correspondiente a CAM 6, las varillas topografiadas quedan casi alineadas en una única línea a lo largo de la orilla (variación de apenas ~5 m en un eje frente a ~140 m en el otro), lo que deja la homografía numéricamente mal condicionada. **Acción recomendada:** volver a topografiar varillas para estas cámaras repartidas en más de una zona/profundidad de la imagen (no solo a lo largo de la orilla), y recalibrar

desde el paso Marcación de la interfaz web.

4.8 Imágenes de diagnóstico

Las imágenes de la figura 4.2 muestran la reproyección de GCPs sobre las imágenes de referencia, de la campaña de calibración inicial. Los vectores de error se representan magnificados $\times 10$.



(a) CAM 1 — RMSE val. 1.45 m



(b) CAM 2 — RMSE val. 3.45 m



(c) CAM 3 — RMSE val. 2.32 m



(d) CAM 4 — RMSE val. 3.06 m (revisar)



(e) CAM 5 — RMSE val. 1.38 m



(f) CAM 6 — RMSE val. 4.74 m (crítico)

Figura 4.2: Diagnóstico de calibración — reproyección de GCPs (vectores $\times 10$)

4.9 Comparación con la homografía de alineación (capítulo 5)

Cuadro 4.3: Las dos homografías del sistema, lado a lado

	Calibración (este capítulo)	Alineación masiva (cap. 5)
Dominio	Píxel de la foto $\rightarrow$ UTM (metros)	Píxel de una foto $\rightarrow$ píxel de otra foto (misma cámara)
Entrada	Varillas GCP marcadas a mano (píxel + UTM conocidos)	Puntos SIFT emparejados por FLANN entre dos capturas
Umbral RANSAC	3 m (dominio UTM)	5 px (dominio píxel)
Se calcula	Una vez por imagen de referencia, al calibrar	Una vez por cada imagen nueva del lote, contra la base
Para qué sirve	Georreferenciar la línea de costa detectada (obtener m ² , coordenadas reales)	Corregir el pequeño desplazamiento de la cámara (viento, mantenimiento) para que la calibración siga siendo válida en fotos posteriores

Ambas usan `cv2.findHomography(..., cv2.RANSAC, umbral)` porque el problema matemático subyacente es el mismo (encontrar la transformación proyectiva que mejor explica un conjunto de correspondencias de puntos, descartando las atípicas) — pero los puntos de entrada, el dominio del umbral y el propósito final son distintos, y conviene no confundir un “RMSE” o unos “inliers” de una con los de la otra al leer los logs o la interfaz.

Capítulo 5

Módulo de Alineación Masiva

¿Esto se hace sobre una imagen o sobre todas las de la cámara?

Sobre un **lote elegido por el usuario** (típicamente, todas las capturas nuevas de una cámara desde la última alineación), nunca sobre “todas las imágenes” automáticamente sin elegir. El usuario escoge primero una **imagen base** (referencia) y después qué imágenes del lote alinear contra ella; el sistema procesa el lote entero de una sola vez (tarea en segundo plano) y produce un resultado por imagen. La homografía de alineación **no se reutiliza entre lotes**: cada imagen del lote obtiene su propia $\mathbf{H}$ respecto a la base elegida en ese momento — no hay una única “homografía de alineación de la cámara” persistente, a diferencia de la homografía de calibración del capítulo 4, que sí queda como valor activo por cámara.

5.1 Motivación

El flujo anterior validaba fotogramas de forma **unitaria**: una llamada API por imagen, sin visión global del lote. Esto presentaba tres problemas:

1. **Escalabilidad**: con 24 imágenes/día por 6 cámaras, revisar manualmente cada alineación era inviable.
2. **Detección de deriva**: sin comparativa global era difícil detectar si una cámara había girado ligeramente con el tiempo.
3. **Objetos dinámicos**: SIFT sin restricciones detectaba gaviotas y olas como features, produciendo homografías sesgadas.

5.2 Arquitectura de estados

El módulo implementa una **máquina de estados no-destructiva** de 5 fases:

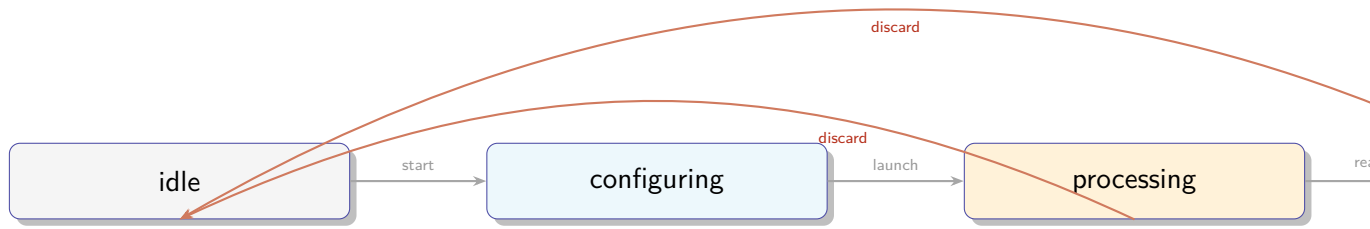


Figura 5.1: Máquina de estados del módulo BatchAlignment

Principio de no-destruictividad

Todo el trabajo intermedio vive en el directorio temporal del sistema operativo (`tempfile.gettempdir()/cv_lit_batch/{job_id}/` — p. ej. `/tmp/...` en Linux/macOS, `C:\Users\...\AppData\Local\Temp\...` en Windows). Los originales de cada cámara **nunca** se modifican. La escritura permanente ocurre **únicamente** al llamar al commit explícito.

5.2.1 Estructura del directorio temporal por job

```

1 {tmp_dir}/cv_lit_batch/{job_id}/
2   aligned/   ← imágenes con warpPerspective aplicado (staging)
3   diff/      ← mapas de delta por imagen
4   blend/     ← blend 50/50 en escala de grises

```

5.3 Pipeline de alineación: algoritmo

5.3.1 Parámetros configurables

Cuadro 5.1: Parámetros del pipeline de alineación (`batch_alignment.py`)

Constante	Valor	Descripción
SIFT_FEATURES	3 000	Keypoints máximos por imagen
LOWE_RATIO	0.75	Ratio test de Lowe para filtrar matches
MIN_INLIERS	15	Inliers mínimos RANSAC (con máscara)
MIN_INLIERS_FULL	30	Inliers mínimos sin máscara (fallback)
RANSAC_THRESH	5.0 px	Umbral de reproyección RANSAC

Vale la pena distinguir claramente los tres números que aparecen en la interfaz de revisión (§5.5), porque cada uno cuenta algo distinto:

1. **Keypoints** (SIFT_FEATURES, hasta 3 000): puntos de la imagen con textura suficientemente distintiva como para reconocerlos en otra foto (esquinas, bordes marcados, estructuras fijas). Se detectan por separado en la imagen base y en cada imagen a alinear.
2. **Matches / coincidencias** (n_good_matches en el resultado): de todos los keypoints de ambas imágenes, cuántos el emparejador FLANN considera que *probablemente* son el mismo punto físico, tras aplicar el ratio test de Lowe (0.75) para descartar coincidencias ambiguas. Todavía puede haber matches erróneos aquí (texturas repetidas, reflejos).
3. **Inliers** (MIN_INLIERS/MIN_INLIERS_FULL): de esos matches, cuántos son geoméricamente consistentes con **una única** homografía — es decir, cuántos sobreviven a RANSAC. Es el número que de verdad importa: la homografía final se calcula solo con los inliers. En la interfaz esto se muestra como “*X / Y puntos*” (inliers / n_good_matches) — un ratio bajo indica que muchos matches iniciales resultaron ser ruido, aunque la alineación en sí haya funcionado bien con los que quedaron.

El δ_{px} (§5.4.3) es un número aparte: no mide cuántos puntos encajaron, sino *cuánto* desplazamiento residual queda de media después de aplicar la homografía — el termómetro de calidad final del resultado, más que del proceso.

5.3.2 Estrategia de dos pasadas

Para combinar la robustez de las máscaras con la resiliencia ante zonas escasas de features:

Lógica de dos pasadas (pseudocódigo)

```

1  # Pasada 1: con máscara de zonas estables
2  H, inliers, n_good, fail = _try_align(gray, ref_kp, ref_des,
3                                     mask=feat_mask,
4                                     threshold=MIN_INLIERS) # 15
5  if inliers >= MIN_INLIERS:
6      result.used_mask = True
7      return H, inliers
8
9  # Pasada 2 (fallback): toda la imagen, umbral más alto
10 H, inliers, n_good, fail = _try_align(gray, ref_kp_full, ref_des_full,
11                                     mask=None,
12                                     threshold=MIN_INLIERS_FULL) # 30
13 result.used_mask = False

```

5.3.3 Flujo completo del pipeline

1. Carga imagen base → extrae keypoints con SIFT (con y sin máscara).
2. Por cada imagen del lote (tarea de fondo asíncrona):
 - a) Detecta features → matching FLANN con ratio test de Lowe (0.75).
 - b) RANSAC → homografía **H** (dos pasadas).
 - c) cv2.warpPerspective → imagen alineada → aligned/.

- d) `_compute_diff_map` → mapa de delta → `diff/`.
 - e) `_blend_grayscale` → `blend 50/50` → `blend/`.
 - f) `_compute_mean_shift` → métrica escalar δ_{px} .
 - g) Actualiza `job.progress_current`.
3. `job.status` = "ready" al finalizar.

5.4 Renderizado del mapa de delta

El mapa de delta es una imagen compuesta de tres capas superpuestas que proporciona tanto información visual intuitiva como métricas cuantitativas.

5.4.1 Capa 1 — Heatmap de diferencia absoluta

$$\Delta_{\text{abs}}(i, j) = \frac{1}{3} \sum_{c \in \{R, G, B\}} |I_{\text{ref}}(i, j, c) - I_{\text{ali}}(i, j, c)| \quad (5.1)$$

Se normaliza y se aplica `COLORMAP_HOT` de OpenCV:

Cuadro 5.2: Leyenda del heatmap HOT

Color	Significado
Negro	$\Delta = 0$ — alineación perfecta
Rojo/naranja	Error de intensidad moderado
Amarillo	Error alto
Blanco	Error severo — zona mal alineada

5.4.2 Capa 2 — Divergencia de bordes (overlay cian)

```

1 edges_ref = cv2.Canny(ref_gray, 40, 120)
2 edges_ali = cv2.Canny(aligned_gray, 40, 120)
3 edge_diff = cv2.dilate(cv2.absdiff(edges_ref, edges_ali), kernel_5x5)
4 composite[edge_diff > 0] = (255, 255, 0) # cian en BGR

```

Los bordes estructurales que **no coinciden** entre referencia y alineada aparecen en **cian**, identificando exactamente qué objeto se desplazó y en qué dirección.

5.4.3 Capa 3 — Métrica escalar δ_{px} (flujo óptico Farneback)

$$\delta_{\text{px}} = \text{mediana} \left(\sqrt{v_x(i, j)^2 + v_y(i, j)^2} \right) \quad (5.2)$$

donde (v_x, v_y) es el campo de flujo óptico residual post-warp calculado con Farneback.

¿Qué es *delta* y para qué sirve?

Después de aplicar la homografía (`warpPerspective`), la imagen alineada debería quedar prácticamente superpuesta a la referencia. δ_{px} mide cuánto NO ha quedado superpuesta: para cada píxel se estima cuánto se tendría que mover todavía para encajar perfectamente (flujo óptico residual), y se toma la mediana de esos desplazamientos sobre toda la imagen. Es, en la práctica, el indicador de *calidad del resultado* de la alineación — mientras que los inliers (recuadro anterior) miden la calidad del *proceso* que llevó a calcular esa homografía. Un valor bajo (cerca de 0 px) significa que la imagen alineada encaja bien con la referencia; por encima de 3 px empieza a notarse un desajuste visual, y es el umbral que la interfaz usa para pintar el dato en ámbar en vez de verde.

5.5 API REST del módulo

Cuadro 5.3: Endpoints del router `/api/batch/`

Método	Ruta	Descripción
POST	<code>/start</code>	Encola job + lanza pipeline en background
GET	<code>/ {id} /status</code>	Polling de progreso (fase processing)
GET	<code>/ {id} /results</code>	Lista completa de métricas (fase ready)
GET	<code>/ {id} /preview/original/ {fn} </code>	Imagen original sin transformar
GET	<code>/ {id} /preview/aligned/ {fn} </code>	Imagen alineada en staging
GET	<code>/ {id} /preview/diff/ {fn} </code>	Mapa de delta compuesto
GET	<code>/ {id} /preview/blend/ {fn} </code>	Blend 50/50 en escala de grises
PATCH	<code>/ {id} /approve/ {fn} </code>	Override de aprobación individual
POST	<code>/ {id} /commit</code>	Two-phase commit → <code>CAM_X/aligned/</code>
DELETE	<code>/ {id} </code>	Descarta job y elimina temp dir

5.6 Modelos de dominio

Dataclasses principales (`batch_alignment.py`)

```

1  @dataclass
2  class AlignmentResult:
3      filename:      str
4      status:        str      # "ok" / "failed" / "reference"
5      inliers:       int = 0  # matches que sobrevivieron a RANSAC
6      n_good_matches: int = 0  # matches FLANN antes de RANSAC (inliers es un
    ↪ subconjunto)
7      mean_shift_px: float = 0.0  # delta - ver §\ref{sec:delta}
8      H:            Optional[List] = None  # homografía 3×3 como lista plana

```

```

9     approved:          bool = True    # override de aprobación por imagen
10    fail_reason:       str = ""        #
    ↪ no_features/low_matches/ransac_failed/bad_lighting:*
11    used_mask:         bool = False   # True = pasada con máscara de zonas estables
12    lighting_issue:    str = ""        # overexposed/underexposed/low_contrast/""
13
14    def __post_init__(self):
15        # Una alineación "failed" cae al frame ORIGINAL sin transformar - nunca
16        # debe quedar aprobada por defecto, o el commit la copiaría al
17        # directorio final sin que nadie la haya revisado.
18        if self.status == "failed":
19            self.approved = False
20
21    @dataclass
22    class BatchJob:
23        job_id:          str
24        cam_id:          int
25        base_filename:   str
26        image_filenames: List[str]
27        status:          str = "pending" #
    ↪ pending/processing/ready/committed/failed/discarded
28        results:         Dict[str, AlignmentResult] = field(default_factory=dict)
29        progress_current: int = 0
30        progress_total:  int = 0

```

5.7 Flujo de commit (two-phase write)

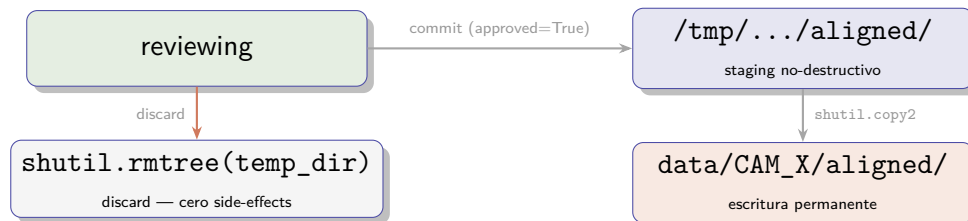


Figura 5.2: Flujo de commit no-destrutivo

Capítulo 6

Máscaras de Zonas Estables SIFT

6.1 Problema: objetos dinámicos en SIFT

Sin restricciones de zona, SIFT detecta features en **cualquier región** de la imagen, incluyendo objetos dinámicos que producen correspondencias erróneas:

- **Gaviotas y pájaros** en vuelo (zona de cielo/horizonte bajo)
- **Olas y espuma** de mar (bordes dinámicos muy contrastados)
- **Personas y bañistas** en la playa
- **Sombras dinámicas** de nubes
- **Hojas de palmeras** agitadas por el viento (ver corrección CAM 3)

Resultado: matches entre objetos en posiciones distintas entre tomas → homografía sesgada → imágenes “giradas” o desplazadas incorrectamente.

6.2 Solución: máscaras de regiones estables

El fichero `calibration/alignment_masks.json` define zonas con **coordenadas fraccionarias** (0,0...1,0) independientes de la resolución real de la imagen:

$$x_{\text{frac}} = \frac{x_{\text{px}}}{W_{\text{imagen}}}, \quad y_{\text{frac}} = \frac{y_{\text{px}}}{H_{\text{imagen}}} \quad (6.1)$$

`alignment_masks.json` - estructura

```
1 {
2   "CAM_1": {
3     "regions": [
4       {"label": "horizonte_mar", "x0": 0.00, "y0": 0.39, "x1": 1.00, "y1":
5         ↪ 0.48},
6       {"label": "arena_baja", "x0": 0.00, "y0": 0.73, "x1": 0.40, "y1":
7         ↪ 0.88},
8       {"label": "vegetacion_dcha", "x0": 0.40, "y0": 0.73, "x1": 1.00, "y1":
9         ↪ 1.00}
10    ]
11  }
```

9 }

6.3 Zonas por cámara

Cuadro 6.1: Configuración de máscaras SIFT por cámara

Cámara	Zona 1	Zona 2	Justificación
CAM 1	Horizonte mar (y=0.39–0.48)	Arena baja + Vegetación dcha ampliada (x=0.40–1.00, hasta fondo)	Excluye cielo con gaviotas
CAM 2	Horizonte (y=0.34–0.48)	Vallas+Vegetación izq (x=0.00–0.28, hasta el borde inferior)	Zona estable con vallas y vegetación fija
CAM 3	Horizonte (y=0.45–0.56)	Kiosco esquina inf. dcha (x=0.87–1.00, y=0.80–0.93)	Estructura fija; excluye las palmeras (x=0.72–0.87) que se mueven con el viento
CAM 4	Horizonte (y=0.38–0.52)	Edificio izq (x=0.00–0.28, hasta el borde inferior)	Edificio como anclaje vertical
CAM 5	Horizonte (y=0.36–0.50)	Zona dcha (x=0.68–1.00)	Zona derecha más estable
CAM 6	Horizonte (y=0.40–0.54)	Edificios izq (x=0.00–0.35)	

6.4 Visualización de zonas estables por cámara

Las figuras siguientes muestran, sobre una imagen real de cada cámara, las regiones activas (coloreadas semitransparentes) donde SIFT extrae keypoints. **Verde** = zona 1 (horizonte/mar), **naranja** = zona 2 (estructura fija secundaria), **cian** = zona 3 (cuando existe). Las regiones sin color son ignoradas por SIFT.

Imagen no disponible en el repositorio:
[mask_cam1.jpg](#)

Figura 6.1: CAM 1 — horizonte del mar ($y = 0,39-0,48$) + arena baja izq + vegetación dcha (hasta fondo). Zona diseñada para evitar las gaviotas que vuelan en $y < 0,38$.

Imagen no disponible en el repositorio:
[mask_cam2.jpg](#)

Figura 6.2: CAM 2 — horizonte del mar ($y = 0,34-0,48$) + zona izquierda con vallas y vegetación ($x = 0,00-0,28$, hasta el borde inferior). Las vallas y masa vegetal ofrecen features estables cuando la visibilidad no permite distinguir el Peñón de Ifach.

Imagen no disponible en el repositorio:
[mask_cam3.jpg](#)

Figura 6.3: CAM 3 — horizonte del mar ($y = 0,45-0,56$) + kiosco de playa en la esquina inferior derecha ($x = 0,87-1,00$, $y = 0,80-0,93$). Se excluye deliberadamente la franja $x = 0,72-0,87$: las palmeras que hay ahí agitan sus hojas con el viento y generaban correspondencias SIFT inestables (rotaciones espurias de más de 100° en las pruebas de alineación).

Imagen no disponible en el repositorio:
[mask_cam4.jpg](#)

Figura 6.4: CAM 4 — horizonte del mar ($y = 0,38\text{--}0,52$) + paseo marítimo/edificios en la esquina izquierda ($x = 0,00\text{--}0,28$, hasta el borde inferior). La infraestructura urbana proporciona features angulares muy discriminantes.

Imagen no disponible en el repositorio:
[mask_cam5.jpg](#)

Figura 6.5: CAM 5 — horizonte del mar ($y = 0,36\text{--}0,50$) + franja derecha estrecha ($x = 0,68\text{--}1,00$). La zona derecha captura vegetación y estructuras fijas con mayor estabilidad que la franja izquierda anterior.

Imagen no disponible en el repositorio:
[mask_cam6.jpg](#)

Figura 6.6: CAM 6 — horizonte del mar ($y = 0,40\text{--}0,54$) + bloque de edificios en la esquina izquierda ($x = 0,00\text{--}0,35$). Los edificios tienen fachadas con ventanas y balcones que generan features muy estables.

Nota sobre CAM_2 — zona vallas_vegetacion

La región etiquetada como **vallas_vegetacion** ($x = 0,00\text{--}0,28$, $y = 0,41\text{--}1,00$) cubre la franja izquierda donde aparecen las vallas y la vegetación costera, que son estructuras estáticas presentes en todas las condiciones de visibilidad. Se extiende hasta el borde inferior de la imagen. En días de alta visibilidad, el Peñón de Ifach puede asomar en esa misma zona como referencia adicional de gran contraste.

6.5 Corrección CAM_1: problema de gaviotas

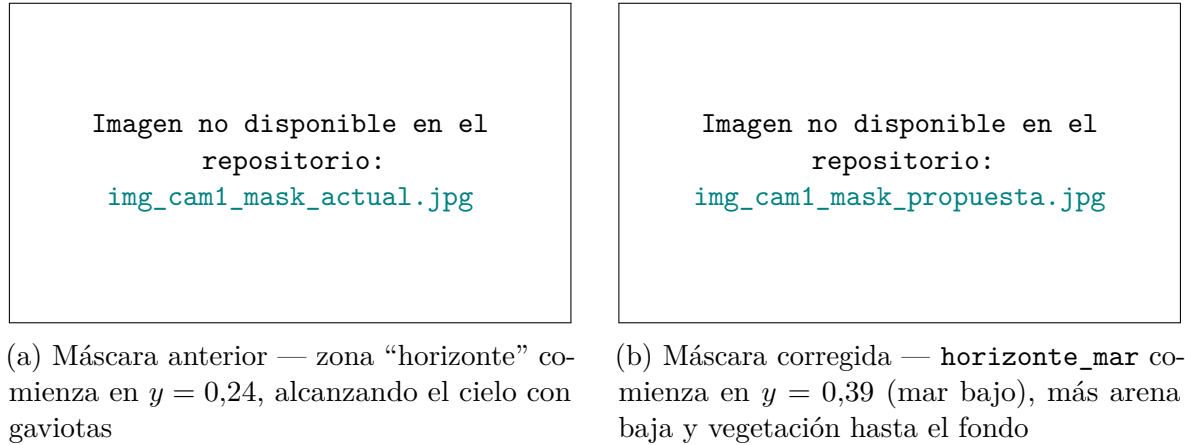


Figura 6.7: Comparativa de máscaras SIFT para CAM 1 (imagen 4608×2682 px)

Por qué CAM_1 es problemática

La cámara apunta hacia el norte sobre la playa. En imágenes de mañana, 5–10 gaviotas sobrevuelan habitualmente en la banda $y \approx 0,14 \dots 0,30$ (cielo bajo). SIFT detecta las aves como features de alto contraste con gran confianza. Al estar en posiciones distintas entre tomas, generan matches cruzados que sesgan la homografía.

La corrección: **horizonte_mar** ($y = 0,39 \dots 0,48$) está en la franja de agua cerca del horizonte, donde los features son reproducibles y estables.

6.5.1 Coordenadas en píxeles (imagen 4608×2682)

Cuadro 6.2: Conversión fracción $\rightarrow$ píxeles para CAM_1

Zona	Esquina sup-izq (px)	Esquina inf-der (px)
horizonte_mar	(0, 1046)	(4608, 1287)
arena_baja	(0, 1957)	(1843, 2360)
vegetacion_dcha	(1843, 1957)	(4608, 2682)

6.6 Cómo ajustar una máscara

1. Abrir una imagen real de la cámara en un visor que muestre coordenadas de píxel.
2. Identificar las regiones estables (sin gaviotas, olas ni personas).
3. Anotar las coordenadas en píxeles de las esquinas.
4. Convertir a fracciones: $x_f = x_{px}/W$, $y_f = y_{px}/H$.
5. Editar `calibration/alignment_masks.json`.

6. Ejecutar el script de verificación:

Script de verificación visual de máscaras

```

1  import cv2, json, numpy as np
2
3  IMG   = "proces_images/data/camera1/IMAGEN_REF.jpg"
4  MASKS = "calibration/alignment_masks.json"
5  CAM   = "CAM_1"
6
7  img = cv2.imread(IMG)
8  h, w = img.shape[:2]
9  overlay = img.copy()
10
11  with open(MASKS) as f:
12      cfg = json.load(f)
13
14  for r in cfg[CAM]["regions"]:
15      x0, y0 = int(r["x0"]*w), int(r["y0"]*h)
16      x1, y1 = int(r["x1"]*w), int(r["y1"]*h)
17      cv2.rectangle(overlay, (x0,y0), (x1,y1), (0,255,0), -1)
18      cv2.putText(overlay, r["label"], (x0+10, y0+30),
19                  cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,0,0), 2)
20
21  result = cv2.addWeighted(img, 0.5, overlay, 0.5, 0)
22  cv2.imwrite("/tmp/mask_check.jpg", result)
23  print(f"Guardado en /tmp/mask_check.jpg ({w}x{h})")

```

Capítulo 7

Interfaz Web

7.1 Arquitectura frontend

La interfaz web está construida con **Vue 3** + **Vite** + **Tailwind CSS**, sin gestión de estado global (no Pinia): cada componente mantiene su propio estado reactivo con `ref()` y `reactive()`.

Para iniciar el entorno de desarrollo:

```
1  # Windows (PowerShell) - workspace de desarrollo vacío en workspace/dev/
2  .\start_dev.ps1
3  # Linux/macOS
4  ./scripts/start_dev.sh
5
6  # Backend:  http://localhost:8000
7  # Frontend: http://localhost:5173
```

7.2 Componente BatchAlignment.vue

Componente principal del módulo de alineación masiva. Integrado en el **paso 3** del flujo de calibración (`Calibration.vue`).

7.2.1 Variables de estado

```
1  const phase    = ref('idle')    //
   ↪ idle/configuring/processing/reviewing/committed
2  const jobId    = ref(null)      // UUID del job backend activo
3  const results  = ref([])        // Array de AlignmentResult del lote
4  const selected = ref(null)      // filename seleccionado en filmstrip
5  const viewMode = ref('diff')    // 'original' | 'aligned' | 'diff' | 'blend'
```

7.2.2 Props y eventos

Cuadro 7.1: Props del componente BatchAlignment

Prop	Tipo	Descripción
camId	Number	ID de cámara (1–6)
imageList	Array	Lista de imágenes disponibles
initialBase	String	Imagen base por defecto (del perfil de calibración)
Emit	Payload	Descripción
notify	(msg, type)	Toast del sistema
committed	{jobId, committed}	Lote confirmado con éxito
discard	—	Usuario descartó el job

7.2.3 Layout de la fase reviewing

Vista de revisión - esquema

1		-----		-----		-----		-----	
2		RESUMEN		[Mapa delta]		[Blend 50/50]		[Alineada]	
3		Aprobadas: 44		-----		-----		-----	
4		Omitidas: 3							
5		Fallidas: 0		ORIGINAL		DELTA / BLEND / ALI			
6		-----							
7		FILMSTRIP							
8		• img_001 OK		-----		-----		-----	
9		▷ img_002 ~2.1px		Leyenda: ■negro=OK ■rojo=error ■cian=bordes					
10		• img_003 FAIL							
11		• img_004 SKIP		inliers: 152 delta: 1.4 px [✓ Aprobada]					
12		...							
13		-----		-----		-----		-----	
14		[Confirmar lote (44)]				[Descartar sin guardar]			
15		-----		-----		-----		-----	

7.2.4 Indicadores del filmstrip

Cuadro 7.2: Código de color en el filmstrip

Color	Estado	Condición
Azul	Referencia	Imagen base seleccionada
Verde	OK	$\delta_{px} \leq 3$
Amarillo	Drift	$\delta_{px} > 3$
Ámbar	Desaprobada	<code>approved = false</code> (manual)
Rojo	Fallida	RANSAC insuficiente

7.2.5 Polling de progreso

El frontend consulta el estado del job cada **800 ms** durante la fase de procesamiento:

```
1 function _poll() {  
2   // GET /api/batch/{jobId}/status  
3   if (data.status === 'ready') {  
4     clearInterval(pollTimer)  
5     await _loadResults() // carga métricas completas  
6     phase.value = 'reviewing'  
7   }  
8   if (data.status === 'failed') {  
9     clearInterval(pollTimer)  
10    emit('notify', data.error, 'error')  
11    phase.value = 'configuring'  
12  }  
13  progress.value = data.progress_current / data.progress_total  
14 }
```

7.3 Flujo completo de uso

1. Dashboard → seleccionar cámara → pestaña *Calibración* → paso 3.
2. Elegir imagen base de referencia del lote.
3. Pulsar “Alinear lote completo” → barra de progreso en tiempo real.
4. Revisar filmstrip: verde = OK, rojo = error.
5. Para cada imagen roja o amarilla: inspeccionar mapa de delta / blend.
6. Desaprobar imágenes concretas (opcional) con toggle en el visor.
7. Pulsar “Confirmar lote” → imágenes aprobadas a `data/{carpeta_cámara}/aligned/`.
8. Las imágenes originales en `data/{carpeta_cámara}/` (p.ej. `camera1/`) quedan **intactas** — `aligned/` es una carpeta hermana, no las sustituye.

Capítulo 8

Región de Interés (ROI)

8.1 Criterios de delimitación

Para optimizar la segmentación con SAM y reducir ruido en la extracción de línea de costa, cada cámara tiene una ROI que excluye:

- **Cielo:** píxeles por encima del horizonte.
- **Infraestructura:** paseos marítimos, espigones, edificios fuera de la zona de interés.
- **Mar profundo:** zona sin rotura de onda donde la línea de costa no es visible.

8.2 Coordenadas por cámara

Las ROIs se almacenan en `calibration/roi_config.json` como un diccionario por cámara con claves `x_min`, `y_min`, `x_max`, `y_max` (píxeles, sobre la resolución nativa de captura de cada cámara — 4608×2682 , no 1920×1080). La tabla 8.1 recoge los valores de producción actualmente configurados.

Cuadro 8.1: ROIs de producción por cámara (`calibration/roi_config.json`)

Cámara	ROI ($x_{min}, y_{min}, x_{max}, y_{max}$) px	Justificación
CAM 1	[0, 993, 4608, 2682]	Excluye horizonte y cielo
CAM 2	[0, 869, 4608, 2682]	Horizonte más bajo
CAM 3	[0, 1118, 4608, 2682]	Ángulo con más cielo visible
CAM 4	[0, 993, 4608, 2682]	Excluye horizonte y cielo
CAM 5	[0, 944, 4608, 2682]	Excluye horizonte y cielo
CAM 6	[0, 1043, 4608, 2682]	Excluye horizonte y cielo

¿De dónde sale un valor por defecto si falta la ROI de una cámara?

Si `roi_config.json` no tiene entrada para una cámara, `analyze_roi()` calcula una por defecto recortando el **40 % superior** de la imagen (`y_min = 0.4 × alto`) — una aproximación razonable al horizonte mientras no se afine a mano, no un

valor definitivo. Los seis valores de la tabla ya están afinados individualmente y sustituyen a ese valor por defecto.

Capítulo 9

Segmentación y Extracción de Línea de Costa

9.1 Segmentación semántica con SAM

Se utiliza **Segment Anything Model (SAM)** de Meta AI en modo *zero-shot*: sin datos de entrenamiento específicos de Guardamar. A diferencia del modo *“automático”* de SAM (que segmenta todo lo que encuentra en la imagen sin guía), **SAMSegmenter** usa **SamPredictor con prompts de punto**: uno o varios puntos semilla por cámara (CAM_PROMPTS, coordenadas fraccionarias sobre la ROI) le dicen a SAM qué región debe segmentar como arena seca, y se toma la máscara con mayor **score** de entre las `multimask_output=True` que devuelve.

Segmentación -- extracto real de SAMSegmenter en `segmentation_sam.py`

```
1 from segment_anything import sam_model_registry, SamPredictor
2
3 sam = sam_model_registry["vit_h"](checkpoint=checkpoint_path)
4 predictor = SamPredictor(sam)
5 predictor.set_image(roi_img_rgb)
6
7 # input_point/input_label vienen de CAM_PROMPTS (puntos semilla por cámara)
8 masks, scores, logits = predictor.predict(
9     point_coords=input_point,
10    point_labels=input_label,
11    multimask_output=True,
12 )
13 mask = masks[np.argmax(scores)] # la máscara con mayor score de las candidatas
```

¿Qué pasa si SAM no está disponible o falla?

`analyze_roi()` (backend/main.py) prueba una cascada de 3 niveles antes de rendirse: **Nivel 1** SAM (si el checkpoint está instalado y no lanza excepción) → **Nivel 2** `color_fallback_segmentation()` (heurística HSV, siempre disponible, no depende de PyTorch ni de descargar el checkpoint de ~2,4 GB) → **Nivel 3** umbral de Otsu sobre el canal V, como último recurso si el color-fallback también devuelve

una máscara vacía. Cada nivel usado queda registrado en el log de actividad (GET /api/logs) para poder auditar con qué método se generó cada resultado.

9.2 Extracción de línea de costa

1. Cálculo de contornos sobre la máscara binaria: `cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`
2. Selección del contorno correspondiente al borde arena húmeda – arena seca.
3. Resultado: polilínea en coordenadas píxel (u, v) .

9.3 Proyección a EPSG:25830

Aplicación de la homografía (4.1) para transformar cada punto (u, v) de la polilínea a coordenadas métricas:

```
pixels_to_utm() - proces_images/georef_export.py

1 def pixels_to_utm(points_px: list | np.ndarray, H: np.ndarray) -> np.ndarray:
2     """points_px: (N,2) pares (col, row). Retorna (N,2) pares (Easting, Northing)
   ↪ en metros."""
3     pts = np.asarray(points_px, dtype=np.float64).reshape(-1, 2)
4     ones = np.ones((len(pts), 1), dtype=np.float64)
5     hom = np.hstack([pts, ones])          # (N, 3)
6     proj = (H @ hom.T).T                  # [X, Y, W]
7     return proj[:, :2] / proj[:, 2:3]     # normalización homogénea
```

9.4 Exportación GeoJSON

```
Estructura real del GeoJSON de salida -- export_geojson() en
georef_export.py

1 {
2     "type": "FeatureCollection",
3     "crs": {
4         "type": "name",
5         "properties": { "name": "urn:ogc:def:crs:EPSG::25830" }
6     },
7     "properties": {
8         "project": "CV-LIT Guardamar",
9         "created": "2026-08-11T17:00:00+00:00",
10        "n_features": 1
11    },
12    "features": [{
13        "type": "Feature",
14        "geometry": {
15            "type": "LineString",
```



```

16     "coordinates": [[711234.5, 4209876.3], ["..."]]
17   },
18   "properties": {
19     "ID_Camara":      "CAM_1",
20     "Timestamp":      "2026-04-30T12:00:00Z",
21     "Confianza_IA":   0.93,
22     "Area_Seca_m2":   14250.7,
23     "RMSE_m":         0.351,
24     "n_puntos":       842,
25     "crs":             "EPSG:25830"
26   }
27 }
28 }
```

¿Por qué el CRS aparece dos veces en formatos distintos?

El bloque `crs` de nivel superior (formato URN, el estándar GeoJSON histórico de QGIS/OGC) identifica el sistema de referencia de la colección completa. El campo `crs": ".EPSG:25830"` dentro de `properties` de cada feature es una copia en formato corto pensada para lectura rápida (p. ej. en la tabla de **Resultados** del frontend) sin tener que parsear el URN — ambos apuntan al mismo sistema, EPSG:25830.

9.5 Índice de confianza (Confianza_IA) y una limitación conocida

`confidence_index()` (en `georef_export.py`) combina cuatro señales en una media ponderada — nunca es un solo número aislado:

1. **Calidad de calibración** (peso 3): $\exp(-\text{RMSE}_m/\text{RMSE_GOOD_M})$ — constante para todas las imágenes de una misma cámara hasta la siguiente calibración.
2. **Cobertura de máscara** (peso 1): fracción del frame detectada como arena seca, con la puntuación máxima entre el 15 % y el 40 % de cobertura.
3. **Probabilidad media de SAM** (peso 2): media del canal "seca" del mapa de probabilidad (`prob_map`), calculada sobre la región de interés completa.
4. **Confianza de los puntos de costa** (peso 2, si aplica): media de la confianza por punto de la línea extraída.

Si el resultado queda por debajo de `confidence_min` (ajustado por cámara en `cam_thresholds.py`, típicamente 0.40–0.50), la imagen se rechaza (`reject_reason: "low_confidence:X.XX"`) y no se guarda en el histórico.

Limitación conocida — playas concurridas se rechazan más de lo esperado

Diagnosticado el 2026-08-11 con una ejecución real de Auto Mode (Cámara 1, una semana completa): 62 de 88 imágenes se rechazaron, y en un primer vistazo el patrón horario diario parecía marea alta. Al inspeccionar las imágenes de verdad, la playa estaba seca y visible — el patrón real es **gente y sombrillas en la**

franja de mayor afluencia, no marea. Se verificó con dos imágenes reales (misma cámara, mismo día): la cobertura de máscara apenas cambia (0.86 vs 0.90), pero la probabilidad media de SAM (factor 3) cae de 0.20 a 0.11 con playa concurrida — eso explica ~80 % de la diferencia de confianza entre ambas.

Se probaron dos alternativas a la media cruda (mediana, y una media recortada por percentil) y ninguna sirve: la mayoría de la región de interés ya es correctamente "no arena"(mar), así que esas estadísticas colapsan sin señal útil. Una media restringida a píxeles por encima de un umbral absoluto de probabilidad tampoco sirve — invierte el problema y premia a la imagen concurrida (un parche pequeño pero muy seguro de arena libre entre sombrillas puntúa mejor que una playa grande y difusa). Con solo dos imágenes de prueba no hay base suficiente para validar una fórmula nueva que afecta a las seis cámaras en producción, así que se ha dejado documentado en vez de forzar un cambio sin validar. Dos caminos razonables para una futura ronda: (a) recopilar más ejemplos reales de playas concurridas/vacías antes de tocar `confidence_index()`, o (b) atacar la causa raíz enmascarando personas antes de segmentar en vez de ajustar la fórmula de confianza a ciegas.

Capítulo 10

Validación y Métricas de Calidad

10.1 Estrategia de validación

Dos niveles de validación: interna (automática) y de campo (independiente)

Conviene distinguir dos cosas que comparten vocabulario (“RMSE”) pero no son lo mismo:

- **Validación interna**, implementada y automática: en el paso Validación de la calibración (capítulo 4), el sistema compara la homografía ajustada contra las *mismas* varillas usadas para calcularla, y expone un RMSE en metros frente a un umbral objetivo (RMSE_GOOD_M = 2 m). Es el criterio de aceptación que se usa hoy en producción.
- **Validación de campo**, metodológica (no automatizada en la interfaz): comparar la línea de costa detectada contra puntos GNSS de campo del IEL que **no** hayan participado en la calibración — la referencia más honesta posible, porque no puede estar sesgada por los mismos puntos que ajustaron la homografía. Es el procedimiento que describe el resto de este capítulo, pensado para auditorías periódicas de precisión más que para el día a día del sistema.

Comparación de la línea detectada contra **transectos GNSS independientes** del IEL: puntos **no usados** en la calibración de homografía.

10.2 Métricas

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N d_i^2} \quad \text{MAE} = \frac{1}{N} \sum_{i=1}^N d_i \quad (10.1)$$

donde d_i es la distancia mínima en metros del punto GNSS i a la polilínea detectada.

10.3 Criterios de aceptación

Cuadro 10.1: Umbrales de calidad del sistema

Métrica	Objetivo	Condición
RMSE geométrico	< 2 m (RMSE_GOOD_M)	Condiciones estándar (12:00h, sin interferencias) — mismo umbral que usa la validación interna del paso Cálculo (§4.5 del capítulo 4)
MAE geométrico	$< 1,0$ m	Objetivo secundario
Cobertura temporal	≥ 90 %	Imágenes a 12:00h procesadas sin error
δ_{px} alineación	< 3 px	Drift residual post-homografía

10.4 Factores de error a monitorizar

- **Calidad de imagen:** niebla, reflejo solar, lluvia, espuma excesiva.
- **Nivel de marea:** afecta la posición real de la línea (calibrar en diferentes estados de marea).
- **Deriva de cámara:** movimiento del soporte por viento, mantenimiento u otras causas.
- **Precisión GCPs:** exactitud del levantamiento GNSS del IEL (instrumental + propagación de error).

10.5 Validación en QGIS

1. Configurar CRS del proyecto en **EPSG:25830**.
2. Cargar GCPs del IEL como capa de puntos.
3. Arrastrar los GeoJSON de `data/processed/lines/` al proyecto.
4. Usar “Distancia a la polilínea más cercana” para calcular d_i .
5. Comparar con ortofoto PNOA (WMS/WMTS del IGN) para validación visual.

Capítulo 11

Módulo de Automatización (Auto Mode)

11.1 ¿Qué problema resuelve?

Sin este módulo, procesar un lote nuevo de capturas de una cámara requiere repetir a mano, imagen a imagen, tres pasos ya vistos en capítulos anteriores: descargar las fotos nuevas de Obscape, alinearlas contra una referencia (capítulo 5) y analizarlas para extraer la línea de costa (capítulo 9). *Auto Mode* automatiza esas tres cosas de punta a punta para un rango de fechas elegido por el usuario, sin reimplementar ninguna: las orquesta, llamando directamente a las mismas funciones que usa el flujo manual.

¿Sobre qué se ejecuta — una imagen / un lote / toda la cámara?

Sobre un **lote acotado por fecha**, elegido explícitamente por el usuario (cámara + rango *desde/hasta*) — nunca se descargan ni procesan imágenes fuera de ese rango, y nunca se lanza automáticamente sin que el usuario pulse *Iniciar*. Dentro de ese rango, se procesan **todas** las imágenes nuevas encontradas (las que ya existen en disco se saltan). No sustituye a la calibración (capítulo 4), que sigue siendo un paso manual previo y obligatorio.

11.2 Requisito previo: la cámara debe estar ya calibrada

Auto Mode analiza imágenes con `analyze_roi()`, el mismo endpoint que usa el botón manual *Analizar imagen* — y ese endpoint necesita una homografía activa para la cámara (capítulo 4, §4.2). Por eso, al pulsar *Iniciar* el backend comprueba primero si existe `calibration/cam_{id}_H.npy` o un H guardado en el perfil de la cámara; si no, rechaza la petición con un mensaje explícito en vez de descargar imágenes que luego no se podrían georreferenciar:

```
_is_calibrated() -- backend/auto_mode.py  
1 def _is_calibrated(cam_id: int) -> bool:  
2     if os.path.exists(f"{CALIBRATION_DIR}/cam_{cam_id}_H.npy"):
```

```

3     return True
4     profile = load_json(f"{CALIBRATION_DIR}/cam_{cam_id}_profile.json")
5     return bool(profile and profile.get("H"))

```

La calibración inicial de una cámara **no** se automatiza: depende de coordenadas UTM de varillas topografiadas en campo, que no vienen de la API de imágenes de Obscape. Auto Mode empieza donde termina la calibración, no la sustituye.

11.3 Las cuatro fases

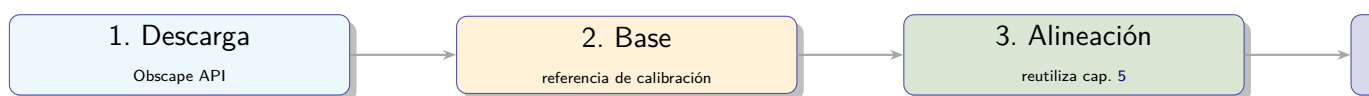


Figura 11.1: Pipeline de Auto Mode — cada fase reutiliza código ya existente, sin reimplementar nada

1. **Descarga:** pide a la API de Obscape los puntos de datos de la cámara en [desde, hasta] y descarga las imágenes que aún no existen en disco (mismo criterio de deduplicación que el descargador manual, `acces_api/obscape_api.py`), guardándolas planas en `DATA_DIR/{carpeta_cámara}/` con el nombrado `<epoch>_<AAAAMDD>_<HHMMSS>_<serie>.jpg` que el resto del backend ya reconoce. Si no hay imágenes nuevas en el rango, el job termina ahí con un mensaje claro (nada que hacer) en vez de seguir con pasos vacíos.
2. **Selección de base:** TODO el lote se alinea contra la **reference_image persistente** de la cámara — la misma imagen que fija `/set-reference` y usa por defecto el flujo manual (capítulo 5), *nunca* la imagen más antigua de este lote concreto. Es un cambio deliberado de 2026-08-11: si cada lote semanal se alineara contra su propia base, cada ejecución quedaría en un marco de píxeles distinto al que se usó para calcular la homografía `H` (que se computa una sola vez, sobre `calibrated_image`) — la `H` fija aplicada a un marco de píxeles distinto introduce un desplazamiento sistemático que ni el RMSE de calibración (solo se calcula una vez) ni la confianza de SAM detectan. Si la cámara no tiene **reference_image** guardada (calibrada sin pasar nunca por `/set-reference`), cae al comportamiento anterior: la imagen más antigua del propio lote, y un lote de una sola imagen no se alinea (nada dentro del lote contra lo que alinearla).
3. **Alineación:** construye un `BatchJob` igual que el flujo manual de alineación masiva y llama directamente a `batch_alignment._run_pipeline()` y, si el resultado queda **ready**, a `commit_batch()` — sin pantalla de revisión intermedia (§11.4 explica por qué es seguro comitear sin que un humano revise cada imagen).
4. **Análisis:** para cada imagen del lote (la alineada si el commit tuvo éxito, la original si no — `_resolve_camera_image_path()` ya resuelve esa prioridad, capítulo 5), llama a `main.analyze_roi()`: la misma función que ejecuta el botón manual *Analizar imagen*. Segmenta, extrae la línea de costa, la proyecta a UTM con la homografía activa de la cámara y guarda el GeoJSON resultante.

11.4 Por qué el commit automático es seguro sin revisión humana

El módulo de alineación masiva (capítulo 5) se diseñó para uso interactivo: un humano revisa cada imagen alineada y aprueba o descarta antes de comitear. Auto Mode no tiene ese paso — comitea en cuanto el job de alineación termina. Esto es seguro porque `AlignmentResult` fuerza `approved=False` automáticamente en cualquier resultado "failed" (ver `__post_init__` en el capítulo 5), y `commit_batch()` solo copia al directorio final las imágenes aprobadas: una alineación fallida queda excluida del commit por sí sola, sin que nadie tenga que marcarla a mano, y el análisis posterior usa entonces la imagen **original sin transformar** para esa captura en concreto (en vez de una alineación mal hecha).

11.5 Modelos de dominio

Dataclasses principales (backend/auto_mode.py)

```

1  @dataclass
2  class ImageResult:
3      filename: str
4      captured_at: Optional[str] = None
5      is_base: bool = False
6      align_status: str = "" # "" / "ok" / "failed" / "reference"
7      confidence: float = 0.0
8      dry_area_m2: float = 0.0
9      rejected: bool = False
10     reject_reason: str = ""
11     error: str = ""
12
13  @dataclass
14  class AutoJob:
15     job_id: str
16     cam_id: int
17     from_date: str
18     to_date: str
19     status: str = "pending" #
20     ↪ pending/downloading/aligning/analyzing/done/error
21     step: str = "" # descripción legible del paso actual
22     progress_current: int = 0
23     progress_total: int = 0
24     downloaded: List[str] = field(default_factory=list)
25     results: List[ImageResult] = field(default_factory=list)
26     error_msg: str = ""

```

11.6 API REST del módulo

Cuadro 11.1: Endpoints del router `/api/automode/`

Método	Ruta	Descripción
POST	<code>/start</code>	Valida cámara calibrada y encola el pipeline completo en background
GET	<code>/_{id}/status</code>	Polling de progreso (fase actual + contadores)
GET	<code>/_{id}/results</code>	Resultado detallado por imagen, solo cuando <code>status</code> es <code>done</code> o <code>error</code>

Igual que el módulo de alineación masiva, el estado de cada job vive **solo en memoria** (`_jobs`, diccionario global) — no persiste entre reinicios del backend, y el frontend hace polling a `/status` cada 1,5 s mientras el job está en curso.

11.7 Nota de empaquetado: import dinámico y PyInstaller

`_download_new_images()` importa `obscape_api` dentro de la propia función, no como import de módulo al principio del fichero (el mismo patrón que usan `segmentation_sam` y compañía en `main.py`, capítulo 3). El análisis estático de PyInstaller no sigue ese tipo de import, así que sin declararlo explícitamente en `hiddenimports` (`backend/desktop_launcher.spec`) Auto Mode habría funcionado perfectamente en desarrollo y fallado con `ModuleNotFoundError` en el `.exe` empaquetado — un fallo que solo se ve al probar el instalador final, no en desarrollo. Corregido añadiendo `."obscape_api"` a `hiddenimports` y `acces_api/` a `pathex` en el `.spec`.

Capítulo 12

Estado del Proyecto y Roadmap

12.1 Estado actual (agosto 2026)

El proyecto se planteó originalmente con una duración de 6 meses (enero–junio 2026, capítulo 1); en la práctica el desarrollo se extendió hasta agosto de 2026 para completar Auto Mode, el instalador de escritorio y una ronda de pulido orientada a producción que no estaba en el alcance original.

Cuadro 12.1: Estado de hitos del proyecto

Mes	Objetivo	Estado	Nota
1	Preparación datos + diseño interfaces	Hecho	Completado
2	Calibración geométrica	Hecho	6 perfiles generados
3	Segmentación (prototipo)	Hecho	SAM funcional
4	Extracción línea + georreferenciación	Hecho	GeoJSON en EPSG:25830
5	Validación y robustez	Hecho	Las 6 cámaras calibradas y validadas
6	Integración y entrega	Hecho	Instalador de escritorio generado y probado

12.2 Completado en la iteración de junio 2026

- Módulo de alineación masiva no-destructiva (`batch_alignment.py`)
 - Router FastAPI `/api/batch/` con 10 endpoints
 - Pipeline async SIFT+FLANN+RANSAC sobre lote completo
 - Mapa de delta compuesto (heatmap HOT + edge-diff cian + métrica Farneback)
 - Two-phase commit: staging `/tmp/` → escritura permanente sólo en commit
- Interfaz de validación por lotes (`BatchAlignment.vue`)
- Corrección de máscaras SIFT para CAM 1 (problema de gaviotas)
- Integración en paso 3 del flujo de calibración (`Calibration.vue`)

- Documentación completa (esta memoria)

12.3 Fase 3 — Automatización y robustez (completada)

1. **Procesamiento por lotes:** resuelto por **Auto Mode** (`backend/auto_mode.py`, capítulo 11) — descarga, alinea y analiza un rango de fechas completo de una cámara de punta a punta, bajo demanda.
2. **Filtros de calidad:** `analyze_roi()` rechaza automáticamente imágenes con confianza de SAM por debajo del umbral configurado (`conf_min`), registrando el motivo en el log de actividad.
3. **Persistencia GeoJSON:** `_append_history()` acumula cada análisis en `coastline_history_cam` consumido por el gráfico "Tendencia Histórica" del Dashboard (`GET /api/historical-data`).
4. **Recalibración de las 6 cámaras:** completada — todas por debajo del objetivo de $RMSE < 2$ m (`RMSE_GOOD_M`, capítulo 4).

12.4 Fase 4 — Entrega e integración (completada)

1. Dashboard de histórico: gráfica de evolución del área seca ya en producción (`Dashboard.vue` + `GET /api/historical-data`).
2. Exportación de informes: `GET /api/report` (CSV/JSON) desde el Dashboard, más un informe PDF individual por cámara con el resumen de su calibración (`GET /api/cameras/{id}/calibration-report.pdf`, capítulo 4).
3. Manual de usuario: `docs/user_manual.md`, con la guía paso a paso del asistente de calibración de 7 pasos y de Auto Mode.
4. Instalador de escritorio para Windows: `installer/cv-lit.iss` + `backend/desktop_launcher.sp` compilado y probado (`installer/output/LineaDeCosta-Setup.exe`) — ver capítulo 14.

12.5 Ronda de pulido para producción (julio–agosto 2026)

Tras cerrar las Fases 3 y 4, una auditoría completa del proyecto orientada a llevarlo a producción añadió, sin estar en el roadmap original:

- Endurecimiento de seguridad: `CORS allow_credentials=False`, validación de PUT en varios endpoints, sanitización de nombres de fichero (`_safe_filename()`), límite mínimo de subida (`MIN_UPLOAD`), y eliminación de credenciales de Obscape hardcodedas (ahora por variables de entorno, `acces_api/obscape_api.py`).
- Eliminación de código y scripts no usados (rutas de alineación duplicadas, herramientas de calibración de un solo uso, la vista rectificada UTM nunca terminada).
- Aviso de geometría inestable en el cálculo de homografía (`geometry_warning`, capí-

tulo 4) y su indicador correspondiente en el stepper del asistente.

- Suite de tests ampliada: `tests/test_auto_mode.py` y `tests/test_geometry_warning.py`, cubriendo Auto Mode y el aviso de geometría sin depender de la red real de Obscape.
- Esta memoria: reescrita y ampliada de forma sustancial (capítulos 4, 5, 11 y auditoría de coherencia del resto de capítulos).

Capítulo 13

Guía de Operación

13.1 Inicio rápido

Arrancar el sistema completo

```
1 # Windows (PowerShell) - activa venv/ y levanta backend+frontend con un
2 # workspace de desarrollo vacío en workspace/dev/ (no toca datos reales)
3 .\start_dev.ps1
4
5 # Linux/macOS (o si se prefiere Conda: ver environment.yml)
6 source venv/bin/activate # o: conda activate cv-lit
7 ./scripts/start_dev.sh
8
9 # Backend: http://localhost:8000 (FastAPI + docs en /docs)
10 # Frontend: http://localhost:5173 (Vue 3 + Vite)
```

13.2 Descarga de imágenes

```
1 # Descarga estándar (últimas 2 semanas, 12:00h)
2 python3 acces_api/scheduled_download.py
3
4 # Diagnóstico de estado de cámaras
5 python3 scripts/verify_setup.py
```

13.3 Calibración

Ya no hay herramienta de calibración por línea de comandos

Las herramientas CLI que existían anteriormente (`calibration_tool.py`, `visualizar_calibracion.py`, `recalibrate.py`) se eliminaron durante el endu-recimiento del proyecto para producción: quedaban duplicadas con el asistente web y sin mantenimiento. La calibración se hace exclusivamente desde la interfaz web, pestaña *Calibración* — asistente de 7 pasos (Vista general → Imágenes →

Alineación → Catálogo → Marcación → Cálculo → Validación), detallado en el capítulo 4. Verificar el motor matemático de la homografía se hace ahora con `python -m pytest tests/test_homography_math.py tests/test_geometry_warning.py`.

13.4 Alineación masiva (CLI)

Para usar el módulo de alineación sin interfaz web:

```

1  # Alinear todas las imágenes de una carpeta respecto a la primera
2  python3 homography/align_images.py \
3      --input /ruta/imagenes/ \
4      --output /ruta/salida/ \
5      --cam 1                      # aplica máscaras SIFT de CAM_1
6
7  # Especificar imagen de referencia manualmente
8  python3 homography/align_images.py \
9      --input /ruta/imagenes/ \
10     --output /ruta/salida/ \
11     --ref imagen_base.jpg \
12     --cam 2

```

Salidas:

- /output/aligned/ — imágenes alineadas
- /output/diagnostics/ — visualizaciones de matches
- /output/homographies.csv — métricas y matrices H por imagen

13.5 Pipeline de extracción

```

1  # Prototipo completo: ROI + Segmentación SAM + Línea de costa
2  python3 proces_images/test_mes3_pipeline.py --cam 1

```

13.6 Operación del día a día: Auto Mode

Para el uso operativo normal (procesar imágenes nuevas de una cámara ya calibrada) no hace falta ninguno de los pasos anteriores: pestaña *Modo automático* de la interfaz web, elegir cámara + rango de fechas y pulsar *Iniciar procesamiento automático*. El sistema descarga, alinea y analiza todo el lote sin intervención manual (capítulo 11 explica el pipeline interno). Requisito: la cámara debe tener ya al menos una calibración calculada — si no, el botón queda deshabilitado con el aviso correspondiente.

- El panel *Estado por cámara* de esa misma pestaña resume de un vistazo qué cámaras están calibradas y con qué RMSE.

- El panel *Actividad de Auto Mode* muestra en tiempo real el log filtrado a solo los mensajes de este módulo (GET /api/logs, prefijo `.^uto Mode`).
- Marcar *Exportar GeoJSON automáticamente al terminar* descarga el GeoJSON de la cámara en cuanto el job pasa a `done`, sin tener que volver a la pantalla.

13.7 Aplicación de escritorio (sin entorno de desarrollo)

Para un operador que no necesita tocar código, el instalador de Windows (`installer/output/LineaDe` capítulo 14) monta la misma interfaz dentro de una ventana nativa (pywebview) sin necesitar terminal, Python instalado ni levantar servidores a mano — el ejecutable arranca el backend en segundo plano y abre la ventana automáticamente.

Capítulo 14

Instalación y Configuración

14.1 Requisitos del sistema

Cuadro 14.1: Requisitos hardware y software

Componente	Requisito mínimo
SO	Linux (Ubuntu 22.04+) o Windows 10+
Python	3.11+
RAM	8 GB (16 GB recomendado para SAM)
GPU	CUDA opcional pero recomendada para SAM
Almacenamiento	50 GB para datos (6 cámaras × 24h/día)
Node.js	18+ (para frontend)

14.2 Instalación del entorno Python

```
1  # venv (recomendado - el mismo que usa start_dev.ps1/start_dev.sh)
2  python -m venv venv
3  venv\Scripts\activate          # Windows
4  source venv/bin/activate       # Linux/macOS
5
6  pip install -r backend/requirements.txt
7
8  # Alternativa: Conda (ver environment.yml en la raíz del repositorio)
9  conda env create -f environment.yml
10 conda activate cv-lit
```

`backend/requirements.txt` instala todo lo necesario (FastAPI, OpenCV, PyTorch, `segment-anything`, `fpdf2` para los informes PDF de calibración, etc.) — no hace falta instalar paquetes sueltos a mano. El checkpoint de SAM (~2,4 GB) no está en `requirements.txt`: se descarga aparte (`backend/download_sam.py`, o `download_sam.exe` en la app empaquetada) porque la app funciona sin él, con segmentación por color/Otsu de respaldo (capítulo 9).

14.3 Instalación del frontend

```

1 cd frontend
2 npm install
3 npm run dev      # modo desarrollo
4 npm run build    # build de producción (dist/)

```

14.4 Configuración de la API Obscape

Las credenciales ya NO se hardcodean en el código

Versiones anteriores de este módulo tenían el usuario y la clave de API escritos directamente en `acces_api/obscape_api.py` — se corrigió durante el endurecimiento de seguridad de esta ronda: ahora `obscape_api.py` las lee de las variables de entorno `OBSCAPE_USERNAME` y `OBSCAPE_API_KEY`, opcionalmente desde un fichero `.env` en la raíz del repositorio (`.env` está en `.gitignore` — nunca se commitea). Si faltan, `ObscapeClient()` lanza un error explícito en vez de fallar en silencio con credenciales vacías.

`.env` en la raíz del repositorio

```

1 OBSCAPE_USERNAME=<usuario del portal Obscape>
2 OBSCAPE_API_KEY=<clave de API>

```

Cuadro 14.2: Parámetros de conexión Obscape

Parámetro	Valor
Base URL	<code>https://obscape.com/portal/api/v3/api</code>
Username / API key	configurados por entorno, ver arriba (no se documentan aquí)

14.5 Generar el instalador de Windows

La app de escritorio se empaqueta en tres pasos, siempre en este orden — el propio `installer/cv-lit.iss` lleva esta misma receta como comentario de cabecera.

1. **Build del frontend** (genera nombres de archivo con hash de contenido nuevos en cada build):

```

1 cd frontend && npm run build && cd ..

```

2. **Empaquetar el backend** con PyInstaller, usando el Python del `venv` del proyecto (más fiable que depender de que `pyinstaller` esté en el `PATH`):

```

1 cd backend

```



```

2  ..\env\Scripts\python.exe -m PyInstaller desktop_launcher.spec --noconfirm
3  ..\env\Scripts\python.exe -m PyInstaller download_sam.spec --noconfirm
4  cd ..

```

3. **Compilar el instalador** con Inno Setup (<https://jrsoftware.org/isinfo.php>, instalable con `winget install -id JRSoftware.InnoSetup -e`):

```

1  "C:\Program Files\Inno Setup 7\ISCC.exe" cv-lit.iss

```

El resultado queda en `installer\output\LineaDeCosta-Setup.exe` (~180 MB — el checkpoint de SAM no va incluido, ver §14).

No recompilar el frontend mientras el paso 2 está en marcha

Condición de carrera real detectada en 2026-08: si se relanza `npm run build` MIENTRAS PyInstaller todavía está empaquetando (por ejemplo, en otra terminal), los nombres de archivo con hash de `frontend/dist/assets/` cambian a mitad de camino y el paquete final contiene referencias a archivos que ya no existen — la app instalada carga en blanco, sin ningún error visible durante el empaquetado (solo un aviso `WARNING: Ignoring non-existent resource` en el log de PyInstaller, fácil de pasar por alto). Si se sospecha que ha pasado, repetir el paso 2 entero sin tocar `frontend/dist/` mientras corre, y comparar que los nombres de archivo en `backend\dist\LineaDeCosta\_internal\frontend_dist\assets\` coincidan exactamente con los de `frontend/dist/assets/`.

Verificación rápida tras compilar, sin necesidad de instalar nada: arrancar el ejecutable empaquetado directamente y comprobar que sirve la app entera.

```

1  cd backend\dist\LineaDeCosta
2  .\LineaDeCosta.exe
3  # en otra terminal:
4  curl http://127.0.0.1:8000/           # debe devolver el HTML del frontend
5  curl http://127.0.0.1:8000/api/cameras # debe devolver JSON con las 6 cámaras

```

Apéndice A

Convenciones de código y Git

A.1 Estrategia de ramas

- **main**: código estable y validado.
- **feature/nombre-modulo**: nuevas funcionalidades.
- **fix/descripcion-error**: correcciones.
- **docs/nombre-doc**: documentación.

Todo cambio debe pasar por Pull Request antes de integrar en **main**.

A.2 Nomenclatura de ficheros

Las imágenes descargadas usan el formato determinista:

`UNIX_YYYYMMDD_HHMMSS_REFERENCIA.jpg`

A.3 Proyección obligatoria

Toda coordenada geográfica en el sistema debe estar en **EPSG:25830**. Nunca almacenar en WGS84 (EPSG:4326) salvo para visualización en APIs de mapas.

Apéndice B

Glosario

EPSG:25830	ETRS89 / UTM zona 30N. Sistema de referencia cartográfico oficial para la Península Ibérica.
GCP	<i>Ground Control Point</i> . Punto de control terrestre con coordenadas conocidas.
Homografía	Transformación proyectiva planar entre dos planos (imagen y terreno).
IEL	Instituto de Ecología Litoral. Suministra los GCPs GNSS del proyecto.
RANSAC	<i>Random Sample Consensus</i> . Estimador robusto de parámetros geométricos.
ROI	<i>Region of Interest</i> . Zona de la imagen relevante para el análisis.
SAM	<i>Segment Anything Model</i> . Modelo de segmentación de Meta AI.
SIFT	<i>Scale-Invariant Feature Transform</i> . Detector de puntos de interés invariante a escala y rotación.
RMSE	<i>Root Mean Square Error</i> . Error cuadrático medio.
MAE	<i>Mean Absolute Error</i> . Error medio absoluto.
Farneback	Algoritmo de flujo óptico denso para estimar campo de desplazamiento entre dos imágenes.
Inliers	Correspondencias que se ajustan al modelo estimado por RANSAC.
Two-phase commit	Patrón de escritura en dos fases: staging temporal → confirmación permanente.
Drift	Desplazamiento residual de píxeles tras la corrección de homografía.
Auto Mode	Módulo que automatiza, para un rango de fechas elegido, la secuencia descarga → alineación → análisis sobre una cámara ya calibrada (capítulo 11).
Aviso de geometría inestable	(<code>geometry_warning</code>) Mensaje que advierte de que RANSAC no encontró un subconjunto mutuamente consistente de varillas suficientemente grande al calcular la homografía — el RMSE resultante puede no reflejar la calidad real de la calibración (capítulo 4).

Varillas usadas

Número de GCPs que entran en el ajuste de la homografía (confirmadas y no excluidas) — **no** es lo mismo que *inliers*, que es cuántas de esas RANSAC clasifica como mutuamente consistentes (capítulo 4).

Apéndice C

Historial de cambios

Cuadro C.1: Historial de versiones del sistema

Fecha	Componente	Cambio
Ene 2026	API client	Cliente Obscape + descarga con deduplicación
Feb 2026	Calibración	<code>calibration_tool.py</code> : intrínsecos + validación ciega
Mar 2026	Calibración	<code>recalibrate.py</code> : recalibración por arrastre de GCPs
Abr 2026	Segmentación	Prototipo SAM + extracción de línea de costa
May 2026	Georreferenciación	Proyección a EPSG:25830 + exportación GeoJSON
May 2026	Calibración	Ejecución real con GCPs del IEL (6 cámaras)
Jun 2026	Backend	<code>batch_alignment.py</code> : pipeline asíncrono + 10 endpoints
Jun 2026	Frontend	<code>BatchAlignment.vue</code> : máquina de estados 5 fases
Jun 2026	Máscaras SIFT	<code>alignment_masks.json</code> : zonas estables por cámara
Jun 2026	CAM 1	Corrección máscara: excluir cielo con gaviotas ($y=0.24 \rightarrow 0.31$)
Jun 2026	Documentación	Esta memoria técnica en $\text{\LaTeX}$
Jul 2026	Backend	<code>auto_mode.py</code> : pipeline automático descarga+alineación+análisis
Jul 2026	Frontend	<code>AutoMode.vue</code> : asistente de cámara + rango de fechas
Jul–Ago 2026	Seguridad	CORS restringido, validación de PUT, credenciales de Obscape movidas a variables de entorno, eliminación de scripts sin mantenimiento (<code>calibration_tool.py</code> , <code>recalibrate.py</code> y otros)
Ago 2026	Calibración	Aviso de geometría inestable (<code>geometry_warning</code>), inliers agregados, histograma de residuos y mapa de cobertura en Validación
Ago 2026	Calibración	Metadatos de perfil + exportación de informe JSON/PDF por cámara
Ago 2026	Marcación	Cota Z opcional, zoom ajustable de la lupa, barra de progreso
Ago 2026	Auto Mode	Exportación automática de GeoJSON, panel de estado por cámara, log filtrado en tiempo real
Ago 2026	Empaquetado	Instalador de escritorio para Windows (<code>LineaDeCosta-Setup.exe</code> , PyInstaller + Inno Setup)
Ago 2026	Tests	<code>test_auto_mode.py</code> , <code>test_geometry_warning.py</code>